

Curso de Go

Aula 15 - Testes em Go

Professor : Antônio Marcelo

NÚCLEA
ACADEMY

 `código_`
BRAZUCA ^[3]





Nessa Aula nos Vamos Ver

- **Como criar testes em Go usando o pacote testing**
- **Como usar testing.T para validar resultados**
- **Como organizar testes em arquivos _test.go**
- **Como criar subtestes com t.Run**
- **Como medir coverage, ou cobertura de testes**
- **Como criar benchmarks com testing.B**
- **Como analisar performance com go test**
- **Como estruturar testes práticos para código real**
- **Parte Prática**



Testes em Go

- Testes servem para verificar se uma função, pacote ou comportamento está funcionando como esperado.
- Em Go, os testes fazem parte da linguagem e não exigem bibliotecas externas.
- O pacote principal usado para testes é:

```
import "testing"
```

- Arquivos de teste seguem esta regra:

```
nome_do_arquivo_test.go
```



Testes em Go - Organização

- Exemplo:

calculadora.go
calculadora_test.go

Arquivo de Teste
package calculadora

```
import "testing"
```

```
func TestSomar(t *testing.T) {  
    resultado := Somar(2, 3)  
    esperado := 5
```

```
    if resultado != esperado {  
        t.Errorf("esperado %d, recebido %d",  
            esperado, resultado)  
    }  
}
```



Código testado
package calculadora

```
func Somar(a int, b int) int {  
    return a + b  
}
```



Para executar o teste

```
go test
```



Estrutura de um teste com testing.T

- Todo teste em Go começa com Test.
- A função precisa receber um ponteiro para testing.T.
- Formato obrigatório:

```
func TestNomeDoTeste(t *testing.T) {  
  
}
```

- O objeto t permite:
- Marcar erro no teste
- Exibir mensagens
- Parar a execução
- Criar subtestes
- Registrar logs

Principais Métodos

```
t.Error("mensagem de erro")  
t.Errorf("valor esperado %d", valor)  
t.Fatal("erro fatal")  
t.Fatalf("erro fatal com valor %d", valor)  
t.Log("mensagem de log")
```




t.Error, t.Errorf, t.Fatal e t.Fatalf

- **t.Error** marca o teste como falho, mas continua executando.
- **t.Fatal** marca o teste como falho e interrompe imediatamente.
- **Regra prática:**
 - Use **t.Error** quando o teste puder continuar
 - Use **t.Fatal** quando não fizer sentido continuar
- **Exemplos**

```
func TestErroSimples(t *testing.T) {  
    resultado := 10  
    esperado := 20  
  
    if resultado != esperado {  
        t.Error("resultado diferente do  
esperado")  
    }  
  
    t.Log("esta linha ainda será  
executada")  
}
```

```
func TestErroFatal(t *testing.T) {  
    resultado := 0  
  
    if resultado == 0 {  
        t.Fatal("resultado não pode ser  
zero")  
    }  
  
    t.Log("esta linha não será executada")  
}
```




t.Error, t.Errorf, t.Fatal e t.Fatalf

- Use Errorf e Fatalf quando quiser formatar mensagens.
- Exemplo

```
func TestComFormatacao(t *testing.T) {  
    resultado := 7  
    esperado := 10  
  
    if resultado != esperado {  
        t.Fatalf("esperado %d, recebido %d",  
esperado, resultado)  
    }  
}
```



Testando funções com retorno de erro

- Em Go, muitas funções retornam valor e erro.
- Exemplo: Código e Teste

```
package divisao

import "errors"

func Dividir(a int, b int) (int, error)
{
    if b == 0 {
        return 0, errors.New("divisão por
zero")
    }

    return a / b, nil
}
```

- Aqui testamos dois cenários:
 - Divisão válida
 - Divisão inválida

```
package divisao

import "testing"

func TestDividirComSucesso(t *testing.T) {
    resultado, err := Dividir(10, 2)

    if err != nil {
        t.Fatalf("erro inesperado: %v", err)
    }

    if resultado != 5 {
        t.Errorf("esperado 5, recebido %d",
resultado)
    }
}

func TestDividirPorZero(t *testing.T) {
    _, err := Dividir(10, 0)

    if err == nil {
        t.Fatal("esperava erro, mas recebeu nil")
    }
}
```




Testes orientados por tabela

- **Testes orientados por tabela são muito usados em Go.**
- **A ideia é criar uma lista de casos de teste e executar todos com o mesmo código.**
- **Vantagens:**
 - **Evita repetição de código**
 - **Facilita adicionar novos cenários**
 - **Deixa o teste mais organizado**
 - **Ajuda a cobrir vários comportamentos**



Testes orientados por tabela - Exemplos

• Código

```
package calculadora
```

```
func Subtrair(a int, b int) int {  
    return a - b  
}
```

• Teste

```
package calculadora
```

```
import "testing"
```

```
func TestSubtrair(t *testing.T) {  
    testes := []struct {  
        nome string  
        a int  
        b int  
        esperado int  
    }{  
        {"subtracao positiva", 10, 3, 7},  
        {"resultado zero", 5, 5, 0},  
        {"resultado negativo", 3, 10, -7},  
    }  
  
    for _, teste := range testes {  
        resultado := Subtrair(teste.a, teste.b)  
  
        if resultado != teste.esperado {  
            t.Errorf(  
                "%s: esperado %d, recebido %d",  
                teste.nome,  
                teste.esperado,  
                resultado,  
            )  
        }  
    }  
}
```



Subtestes com t.Run

- Subtestes permitem dividir um teste maior em partes menores.
- Eles são muito úteis junto com testes orientados por tabela.
- Subtestes ajudam a identificar exatamente qual cenário falhou.
- Executando todos os testes:

```
go test
```

- Executando apenas um subteste:

```
go test -run TestNormalizar/minusculas
```



Subtestes com t.Run - Exemplos

• Código

```
package texto

import "strings"

func Normalizar(nome string) string {
    return
    strings.TrimSpace(strings.ToLower(nome))
}
```

• Teste

```
package texto

import "testing"

func TestNormalizar(t *testing.T) {
    testes := []struct {
        nome string
        entrada string
        esperado string
    }{
        {"minúsculas", "JOAO", "joao"},
        {"espacos", " maria ", "maria"},
        {"misto", " PEDRO ", "pedro"},
    }

    for _, teste := range testes {
        t.Run(teste.nome, func(t *testing.T) {
            resultado := Normalizar(teste.entrada)

            if resultado != teste.esperado {
                t.Errorf("esperado %q, recebido %q", teste.esperado,
                    resultado)
            }
        })
    }
}
```



Coverage, cobertura de testes

- Coverage mostra quanto do código foi executado pelos testes.
- Comando básico:

```
go test -cover
```

- Exemplo de saída:

```
coverage: 85.7% of statements
```

- Para gerar um arquivo de cobertura:

```
go test -coverprofile=coverage.out
```

- Para visualizar no navegador, gera uma saída html

```
go tool cover -html=coverage.out
```




Benchmarks em Go

- Benchmarks medem performance.
- Eles mostram quanto tempo uma função leva para executar.
- Funções de benchmark começam com Benchmark.
- Formato:

```
func BenchmarkNome(b *testing.B) {  
  
}
```

- exemplo benchmark

```
package soma  
  
import "testing"  
  
func BenchmarkSomarLista(b *testing.B)  
{  
    numeros := []int{1, 2, 3, 4, 5}  
  
    for i := 0; i < b.N; i++ {  
        SomarLista(numeros)  
    }  
}
```

- Executar benchmark:

```
go test -bench=.
```

- Saída

```
BenchmarkSomarLista-8 100000000 12.5 ns/op
```

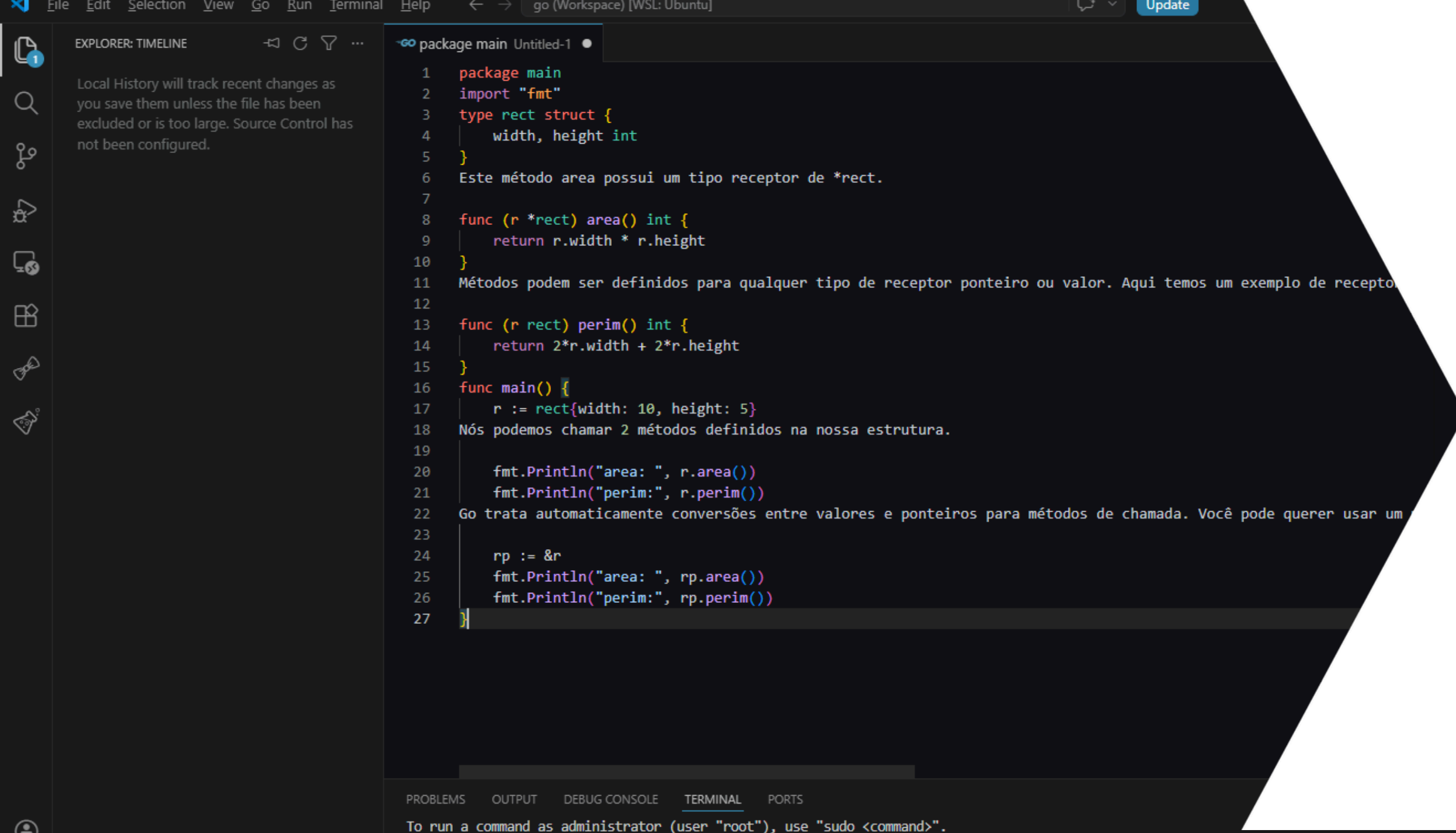
- Significado:

- **BenchmarkSomarLista-8**: nome do benchmark e quantidade de CPUs usada
- **100000000**: quantidade de execuções
- **12.5 ns/op**: tempo médio por operação



Boas práticas

- **Teste comportamento, não implementação**
- **Evite testar detalhes internos.**
- **Teste o resultado esperado.**
- **Use nomes claros**
- **Use testes orientados por tabela**
- **Teste casos de erro ! Não teste só o caminho feliz.**
- **Mantenha testes pequenos Cada teste deve validar uma ideia principal. Testes grandes demais ficam difíceis de entender e manter.**
- **Evite dependências externas nos testes unitários**
- **Banco de dados, API externa e rede tornam o teste lento e instável. Prefira interfaces, mocks ou fakes.**
- **Use coverage com bom senso**
- **Coverage alto ajuda, mas não garante qualidade. Melhor ter 70% de testes bons do que 100% de testes**
- **Use benchmarks só quando fizer sentido. Benchmark é útil para código crítico de performance.**



The screenshot shows a Go IDE with a file named 'Untitled-1' in the 'package main' directory. The code defines a struct 'rect' with 'width' and 'height' of type 'int'. It includes two methods: 'area()' which returns the area, and 'perim()' which returns the perimeter. The 'main' function creates a 'rect' with width 10 and height 5, prints its area and perimeter, and then uses a pointer 'rp' to call the same methods, demonstrating pointer receivers. The IDE interface includes a sidebar with 'EXPLORER: TIMELINE' and a bottom panel with 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL', and 'PORTS'.

```
1 package main
2 import "fmt"
3 type rect struct {
4     width, height int
5 }
6 Este método area possui um tipo receptor de *rect.
7
8 func (r *rect) area() int {
9     return r.width * r.height
10 }
11 Métodos podem ser definidos para qualquer tipo de receptor ponteiro ou valor. Aqui temos um exemplo de recepto
12
13 func (r rect) perim() int {
14     return 2*r.width + 2*r.height
15 }
16 func main() {
17     r := rect{width: 10, height: 5}
18     Nós podemos chamar 2 métodos definidos na nossa estrutura.
19
20     fmt.Println("area: ", r.area())
21     fmt.Println("perim:", r.perim())
22     Go trata automaticamente conversões entre valores e ponteiros para métodos de chamada. Você pode querer usar um
23
24     rp := &r
25     fmt.Println("area: ", rp.area())
26     fmt.Println("perim:", rp.perim())
27 }
```

NÚCLEA
ACADEMY

Parte Prática

Praticar Testes em Go